

GamutX

Conditional Statements

Python Basics

Introduction to Control Flow

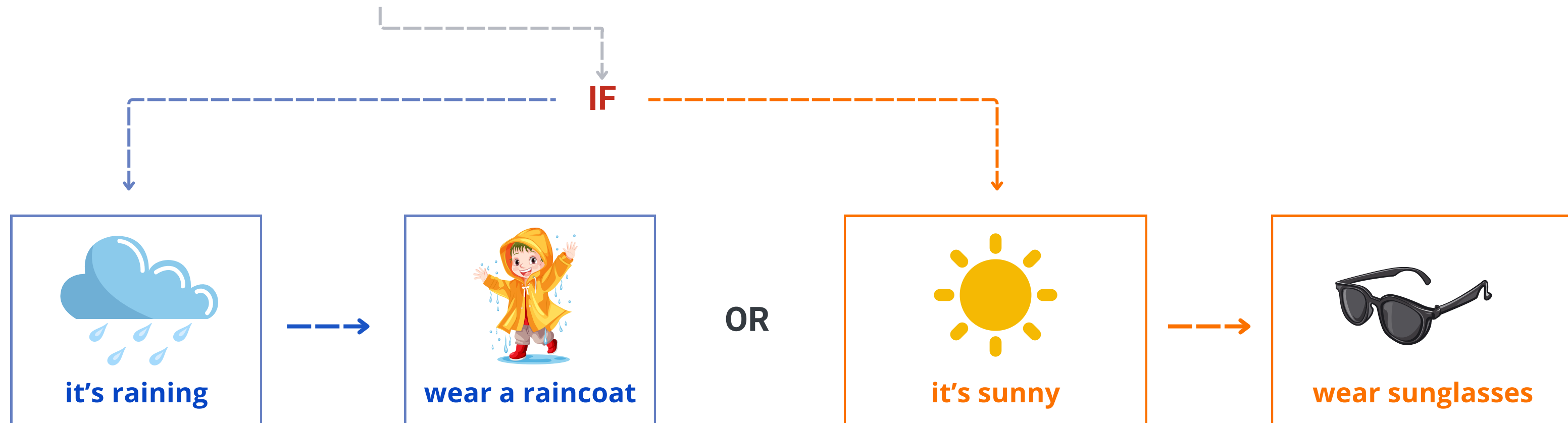
- What is Control Flow?
- What is Indentation?
- How to Check Indentation in VS Code
- How to Set Indentation in VS Code
- Various Control Flow Statements
- Types of Conditional Statements

What is Control Flow?

- Control flow is the part of a program that **decides which code runs and when**.
- Think of it like a **set of instructions** that tells your program what to do based on certain situations.

Real-Life Example: Decision-Making

In everyday life, you **make decisions** all the time., For Example



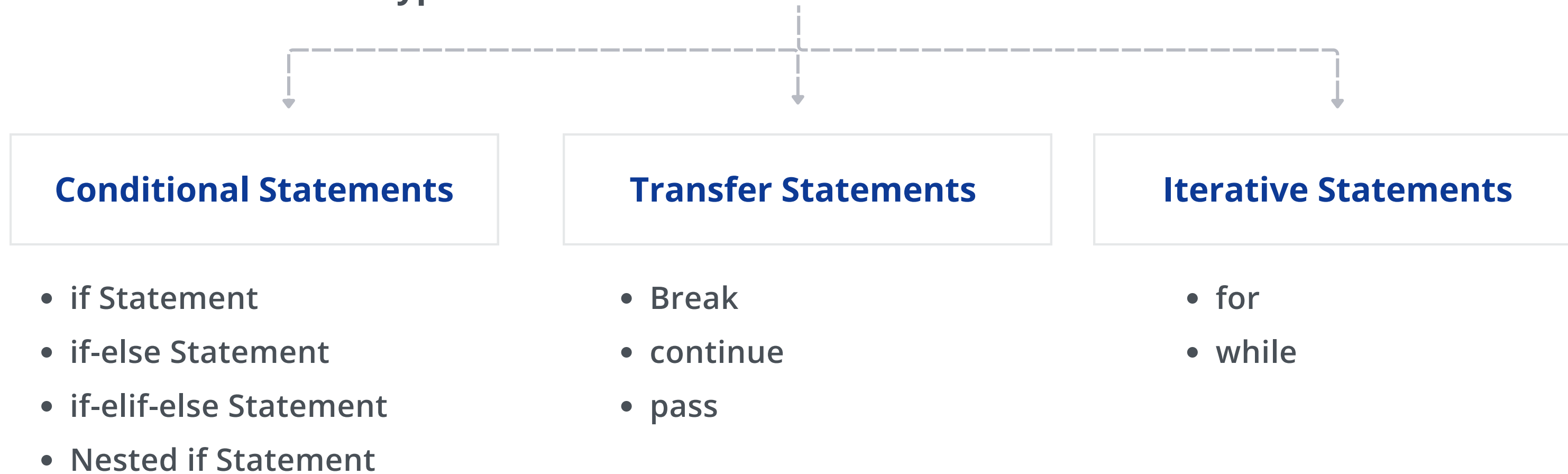
- You're checking the **condition** (**weather**), and then taking an **action** based on it.

Why It Matters

- **Without control flow:**
 - A program would just **run straight through line by line**, doing the same thing every time.
 - You wouldn't be able to make decisions or respond to different inputs or situations.
 - **With control flow:**
 - Your program becomes smarter.
 - It can **make decisions, repeat tasks, and react to changes**.
- 

Various Control Flow Statements

- In Python, control flow statements are **used to control which part of the code runs and when.**
- There are **three main types of control flow statements**



Types of Conditional Statements

- Conditional statements help **a program make decisions based on certain conditions.**
- These include:

if statement

if-else statement

Nested if statements

if-elif-else statementtton (*)

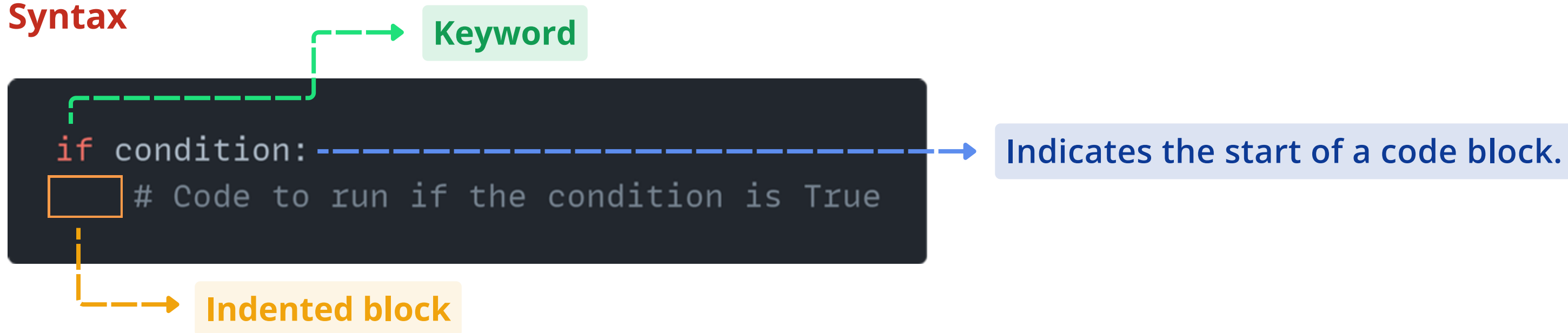
The if Statement

- What is if statement ?
- Understanding - If Statement

What is if statement ?

- The if statement is used to **execute a block of code only if a specified condition is true.**
- If the **condition is True**, it runs the code inside the block.
- If the condition is **False**, it **skips the block**.

Syntax

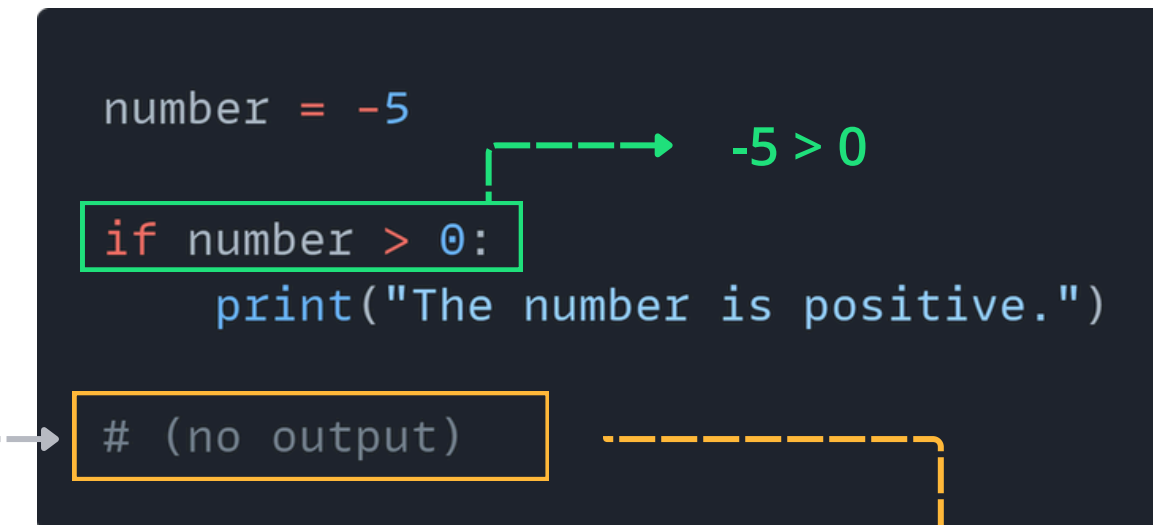


Understanding - If Statement

Check if a Number is Positive



Let's try with a Negative Number



- -5 > 0 is False. So the code inside the if block is skipped.

! Indentation Matters

- Python uses indentation (spaces or a tab) to know which code belongs to the if block.
- If you forget to indent, Python will give an error.

Points to Remember:

- **What the if Statement Does:**
 - You can think of the if statement as asking a yes/no question.
 - Is the number greater than 0?
 - **If yes** → do this
 - **If no** → skip it
- **Indentation Matters:**
 - Python uses indentation (spaces or a tab) to know which code belongs to the if block.
 - If you **forget to indent**, Python will **give an error**.

Concept Practice Task

Checking if a Number is Equal

```
number = 100

if number == 100:
    print("The number is 100.")
```

```
#Output: The number is 100.
```

Checking if a String is "Hello"

```
greeting = "Hello"

if greeting == "Hello":
    print("The greeting is Hello.")
```

```
#Output: The greeting is Hello.
```

Checking if a Number is Greater Than

```
number = 75

if number > 50:
    print("The number is greater than 50.")
```

```
#Output: The number is greater than 50.
```

Check if Temperature is Above Freezing

```
temperature = 10

if temperature > 0:
    print("The temperature is above freezing.")
```

```
#Output: The temperature is above freezing.
```

if-else Statement

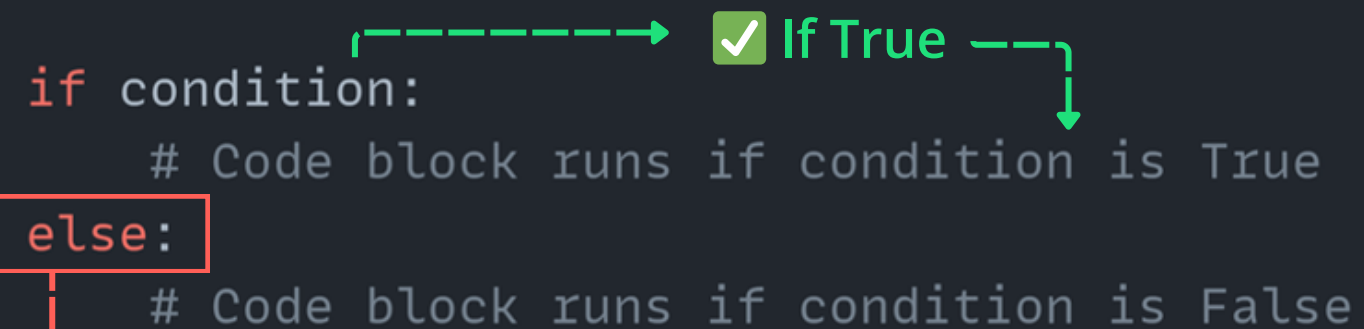
- What is if-else statement?
- Understanding the if-else Statement

What is if-else statement?

- The if-else statement lets us choose between two options in our code.
- we check a condition:
 - If it's **True**, one block of code runs.
 - If it's **False**, a different block runs.

Syntax

```
if condition:
    # Code block runs if condition is True
else:
    # Code block runs if condition is False
```



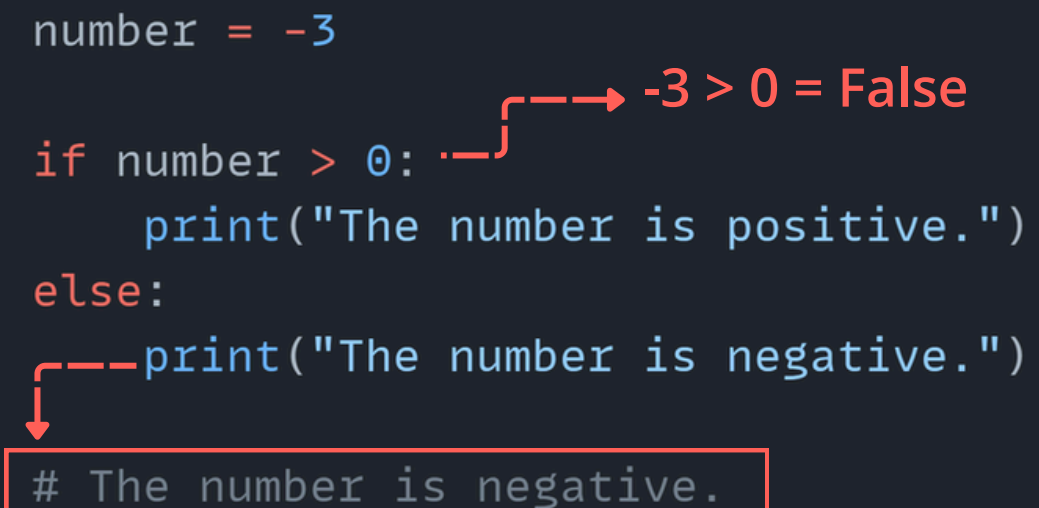
✗ If False → Code inside else runs

- Only one of the two blocks will run — not both.

Example

```
number = -3

if number > 0:
    print("The number is positive.")
else:
    print("The number is negative.")
```



Concept Practice Task

GamutX

Checking if a Number is Even or Odd

```
number = 8

if number % 2 == 0:
    print("The number is even.")
else:
    print("The number is odd.")
```

```
#Output: The number is even.
```

Checking if a String Contains "Python"

```
text = "I love Python."

if "Python" in text:
    print("The text contains 'Python'.")
else:
    print("The text does not contain 'Python'.")
```

```
#Output: The text contains 'Python'.
```

Checking if a Person is Eligible to Vote

```
age = 20

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

```
#Output: You are eligible to vote.
```

Checking if a Temperature is Hot or Cold

```
temperature = 30

if temperature > 25:
    print("It's hot outside.")
else:
    print("It's cold outside.")
```

```
#Output: It's hot outside.
```

if-elif-else Statement

- What is if-elif-else Statement ?
- Understanding - if elif else statement

What is if-elif-else Statement ?

- Use if-elif-else when **you have more than two choices**.
- It helps the program **pick one path out of many**.

Syntax

```
if condition1:  
    # Runs if condition1 is True  
elif condition2:  
    # Runs if condition1 is False and condition2 is True  
elif condition3:  
    # Runs if previous conditions are False and condition3 is True  
else:  
    # Runs if all conditions are False
```

Note

- Only the first True condition runs — rest are ignored.

Example

```
score = 85
```

```
if score >= 90:  
    print("Grade: A")
```

85 >= 90 ❌ No

```
elif score >= 80:  
    print("Grade: B")
```

85 >= 80 ✅ Yes

```
elif score >= 70:  
    print("Grade: C")
```

Skipped

```
else:  
    print("Grade: D")
```

```
# Grade: B
```

Concept Practice Task

Checking Temperature Ranges

```
temperature = 22

if temperature < 0:
    print("It's freezing.")
elif temperature < 10:
    print("It's very cold.")
elif temperature < 20:
    print("It's cool.")
elif temperature < 30:
    print("It's warm.")
else:
    print("It's hot.")
```

```
#Output: It's warm.
```

Checking User Age for Discounts

```
age = 16

if age < 12:
    print("Child ticket: $10")
elif age < 18:
    print("Teen ticket: $15")
elif age < 65:
    print("Adult ticket: $20")
else:
    print("Senior ticket: $12")
```

```
#Output: Teen ticket: $15
```


Concept Practice Task

Check if a Number is Small, Medium, or Large

```
number = 45

if number < 10:
    print("The number is small.")
elif number < 50:
    print("The number is medium.")
else:
    print("The number is large.")
```

```
#Output: The number is medium.
```

Check if a Number is Positive, Negative, or Zero

```
number = float(input("Enter a number: "))

if number > 0:
    print("The number is positive.")
elif number < 0:
    print("The number is negative.")
else:
    print("The number is zero.")
```

Output 1:

- Enter a number: 5
- The number is positive.

Output 2:

- Enter a number: -3
- The number is negative.

Output 3:

- Enter a number: 0
- The number is zero.

Nested if Statements

- What is Nested if Statements ?
- Understanding - Nested if

What is Nested if Statements ?

- A nested if means **putting one if statement inside another**.
- This is useful when we **want to check one condition first**, and then, **based on that, check another**.

Syntax

```
if condition1:  
    if condition2:  
        # Runs if both condition1 and condition2 are True  
    else:  
        # Runs if condition1 is True but condition2 is False  
else:  
    # Runs if condition1 is False
```

🔍 How It Works

- **condition1** is checked first.
- If **condition1** is **True**, then **condition2** is checked.
- If **both** are **True**, the **inner block** runs.
- If **condition1** is **False**, it goes straight to the **outer else**.

Understanding - Nested if

```
weather = "sunny"
has_umbrella = "no"

if weather == "rainy":
    if has_umbrella == "yes":
        print("You can go outside without getting wet.")
    else:
        print("You might get wet. Stay indoors or find an umbrella!")
else:
    print("The weather is nice. You can go outside!")

# Output : The weather is nice. You can go outside!
```

Python checks:

- if weather == "rainy" → **✗ It's False**

↓
— skip inner check

Try Changing the Values

weather = "rainy"

→ has_umbrella = "yes" → You can go outside without getting wet.

→ has_umbrella = "no" → You might get wet. Stay indoors or find an umbrella!

Concept Practice Task

Checking for Student Scholarship Eligibility

```
grade = 88
extra_curricular = True

if grade >= 85:
    if extra_curricular:
        print("Eligible for scholarship.")
    else:
        print("Not eligible for scholarship. ")
else:
    print("Not eligible for scholarship (grade too low).")

# Output: Eligible for scholarship.
```

Checking a Discount Based on Age and Membership Status

```
age = 30
is_member = False

if age >= 25:
    if is_member:
        print("Discount: 20%")
    else:
        print("Discount: 10%")
else:
    print("No discount.")

# Output: Discount: 10%
```

Determining Eligibility for a Special Event

```
is_weekend = True
has_invite = False

if is_weekend:
    if has_invite:
        print("You can attend the event.")
    else:
        print("You cannot attend the event (no invite).")
else:
    print("You cannot attend the event (not a weekend).")

# Output: You cannot attend the event (no invite).
```

Checking for Summer Internship Eligibility

```
gpa = 3.5
completed_course = True

if gpa >= 3.0:
    if completed_course:
        print("Eligible for summer internship.")
    else:
        print("Not eligible for summer internship (course not completed).")
else:
    print("Not eligible for summer internship (GPA too low).")

# Output:
# Eligible for summer internship.
```

Operators and Expressions

- Comparison Operators
- Boolean Expressions

Comparison Operators

- **Comparison operators** are used to compare values. The result of a comparison is a Boolean value (**True** or **False**).

Here's list of comparison operators:

Operator	Meaning	Example
1. ==	→ Equal to	→ a == b
2. !=	→ Not equal to	→ a != b
3. >	→ Greater than	→ a > b
4. <	→ Less than	→ a < b
5. >=	→ Greater than or equal to	→ a >= b
6. <=	→ Less than or equal to	→ a <= b

Using == (Equal to)

```
a = 10
b = 10

if a == b:
    print("a is equal to b")
else:
    print("a is not equal to b")
```

```
#Output: a is equal to b
```

Using != (Not Equal To)

```
x = 15
y = 20

if x != y:
    print("x is not equal to y")
else:
    print("x is equal to y")
```

```
# Output: x is not equal to y
```

Using > (Greater Than)

```
num1 = 25
num2 = 20

if num1 > num2:
    print("num1 is greater than num2")
else:
    print("num1 is not greater than num2")
```

```
# Output: num1 is greater than num2
```

Using < (Less Than)

```
value1 = 30
value2 = 40

if value1 < value2:
    print("value1 is less than value2")
else:
    print("value1 is not less than value2")
```

```
# Output: value1 is less than value2
```


Using >= (Greater Than or Equal to)

```
num_a = 50
num_b = 50

if num_a >= num_b:
    print("num_a is greater than or equal to num_b")
else:
    print("num_a is less than num_b")
```

Output: num_a is greater than or equal to num_b

Using <= (Less Than or Equal to)

```
age = 21
legal_age = 21

if age <= legal_age:
    print("Age is less than or equal to the legal age")
else:
    print("Age is greater than the legal age")
```

#Output: Age is less than or equal to the legal age

Boolean Expressions

- A Boolean expression is an expression that results in either: True, False

Basic Usage

```
is_raining = True

if is_raining:
    print("Remember to take an umbrella!")
else:
    print("No need for an umbrella today.")
```

```
# Output: Remember to take an umbrella!
```

Weather Conditions

```
temperature = 60
is_windy = False

if temperature > 50 and not is_windy:
    print("Good day to go outside!")
else:
    print("It's not ideal to go outside.")
```

```
# Output: Good day to go outside!
```

Multiple Conditions for Special Offer

```
purchased_items = 5
is_member = False

if purchased_items >= 3 and (is_member or purchased_items > 5):
    print("Eligible for the special offer!")
else:
    print("Not eligible for the special offer.")
```

```
# Output: Not eligible for the special offer.
```

is and in Keywords

- Using is
- Using in
- Using “is not”
- Using “not in”

Using is

- The is keyword is used to check if two variables point to the same object in memory.
- This is not the same as ==, which checks if the values are the same.

Using is with Integer

```
a = 1000
b = 1000

if a is b:
    print("Both 'a' and 'b' are the same object in memory.")
else:
    print("They are different objects in memory.")
```

```
# Output:They are different objects in memory.
```

Using is with Strings

```
str1 = "hello"
str2 = "hello"

if str1 is str2:
    print("Both 'str1' and 'str2' are the same object in memory.")
else:
    print("They are different objects in memory.")
```

```
# Output:Both 'str1' and 'str2' are the same object in memory.
```

Concept Practice Task

Using is with Lists

```
a = [1, 2, 3]
b = a

if a is b:
    print("a and b refer to the same object.")
else:
    print("a and b do not refer to the same object.")

# Output: a and b refer to the same object.
```

Comparing Floats

```
f1 = 0.1 + 0.2
f2 = 0.3

if f1 is f2:
    print("Both 'f1' and 'f2' are the same object in memory.")
else:
    print("They are different objects in memory.")

# Output:
# They are different objects in memory.
```

Using is with Dictionaries

```
dict1 = {'name': 'Alice', 'age': 30}
dict2 = {'name': 'Alice', 'age': 30}

if dict1 is dict2:
    print("Both 'dict1' and 'dict2' are the same object in memory.")
else:
    print("They are different objects in memory.")

# Output: They are different objects in memory.
```

Using in

- The in keyword is used to check if a value exists inside a collection — like a string, list, or dictionary.
- This is called a membership test.

Using 'in' with a string

```
my_string = "Hello, World!"

if "World" in my_string:
    print("'World' is in the string.")
else:
    print("'World' is not in the string.")
```

```
# Output: 'World' is in the string.
```

Using in with Lists

```
my_list = [10, 20, 30, 40, 50]

if 30 in my_list:
    print("30 is in the list.")
else:
    print("30 is not in the list.")
```

```
# Output: 30 is in the list.
```

Concept Practice Task

Using in with a Dictionary

```
my_dict = {'name': 'Alice', 'age': 30}

if 'age' in my_dict:
    print("Key 'age' is present in the dictionary.")
else:
    print("Key 'age' is not present in the dictionary.")
```

```
# Output:
# Key 'age' is present in the dictionary.
```

Using in with a List Containing Integers and Floats

```
numbers = [1, 2, 3.5, 4.5, 5]

if 3 in numbers:
    print("3 is in the list.")
else:
    print("3 is not in the list.")

if 3.5 in numbers:
    print("3.5 is in the list.")
else:
    print("3.5 is not in the list.")
```

```
# Output:
# 3 is in the list.
# 3.5 is in the list.
```

Using “is not” and “not in”

Understanding - is not → Checks if two variables do not refer to the same object.

Using is not

```
a = 1000
b = 1000

if a is not b:
    print("The variables 'a' and 'b' do not refer to the same object in memory.")
else:
    print("The variables 'a' and 'b' refer to the same object in memory.")
```

```
# Output:
# The variables 'a' and 'b' do not refer to the same object in memory.
```

Using is not with String

```
str1 = "Hello"
str2 = "World"

if str1 is not str2:
    print("str1 and str2 are different objects in memory.")
```

```
# Output:str1 and str2 are different objects in memory.
```


Concept Practice Task

GamutX

Using is not with List

```
x = [4, 5, 6]
y = [4, 5, 6]

if x is not y:
    print("x and y are not the same object.")
else:
    print("x and y are the same object.")

# Output: x and y are not the same object.
```

Using is not with Dictionaries

```
dict1 = {'name': 'Alice'}
dict2 = {'name': 'Bob'}

if dict1 is not dict2:
    print("dict1 and dict2 are different objects in memory.")
else:
    print("dict1 and dict2 refer to the same object in memory.")

# Output: dict1 and dict2 are different objects in memory.
```


Understanding not in:

- Checks if a value does not exist within a sequence.

Using not in with String

```
text = "Hello, World!"

if "Python" not in text:
    print("The substring 'Python' is not found in the text.")
else:
    print("The substring 'Python' is found in the text.")

# Output:The substring 'Python' is not found in the text.
```

Using not in with List

```
colors = ['red', 'green', 'blue']

if 'yellow' not in colors:
    print("Yellow is not in the list of colors.")
else:
    print("Yellow is in the list of colors.")

#Output:Yellow is not in the list of colors.
```

Using not in with Dictionaries

```
person = {'name': 'Alice', 'age': 25}

if 'gender' not in person:
    print("The key 'gender' is not found in the dictionary.")
else:
    print("The key 'gender' is found in the dictionary.")

# Output:The key 'gender' is not found in the dictionary.
```

Indentation

- What is Indentation?
- How to Check Indentation Level in VS Code
- How to Set Indentation Level in VS Code
- Rules of Indentation in Python

What is Indentation?

- Indentation refers to the **space at the beginning of a line of code**.
- This space tells Python **which lines belong together**, like inside a function or inside a loop.

Types of Indentation

1. Spaces for Indentation

- These are just normal space characters.
- Most common: **4 spaces per level**.
- Recommended by Python's style guide (PEP 8).

2. Tabs for Indentation

- Pressing the **Tab** key creates a tab character.
- A tab usually equals 4 spaces, but it can **look different** depending on editor settings.

🔍 Which One Should You Use?

- It's best to use **spaces**, as recommended by Python's official guidelines.
- Spaces are **more predictable and avoid problems** caused by mixing spaces and tabs.
- **Mixing both can lead to hidden bugs or indentation errors**, especially when sharing code with others.

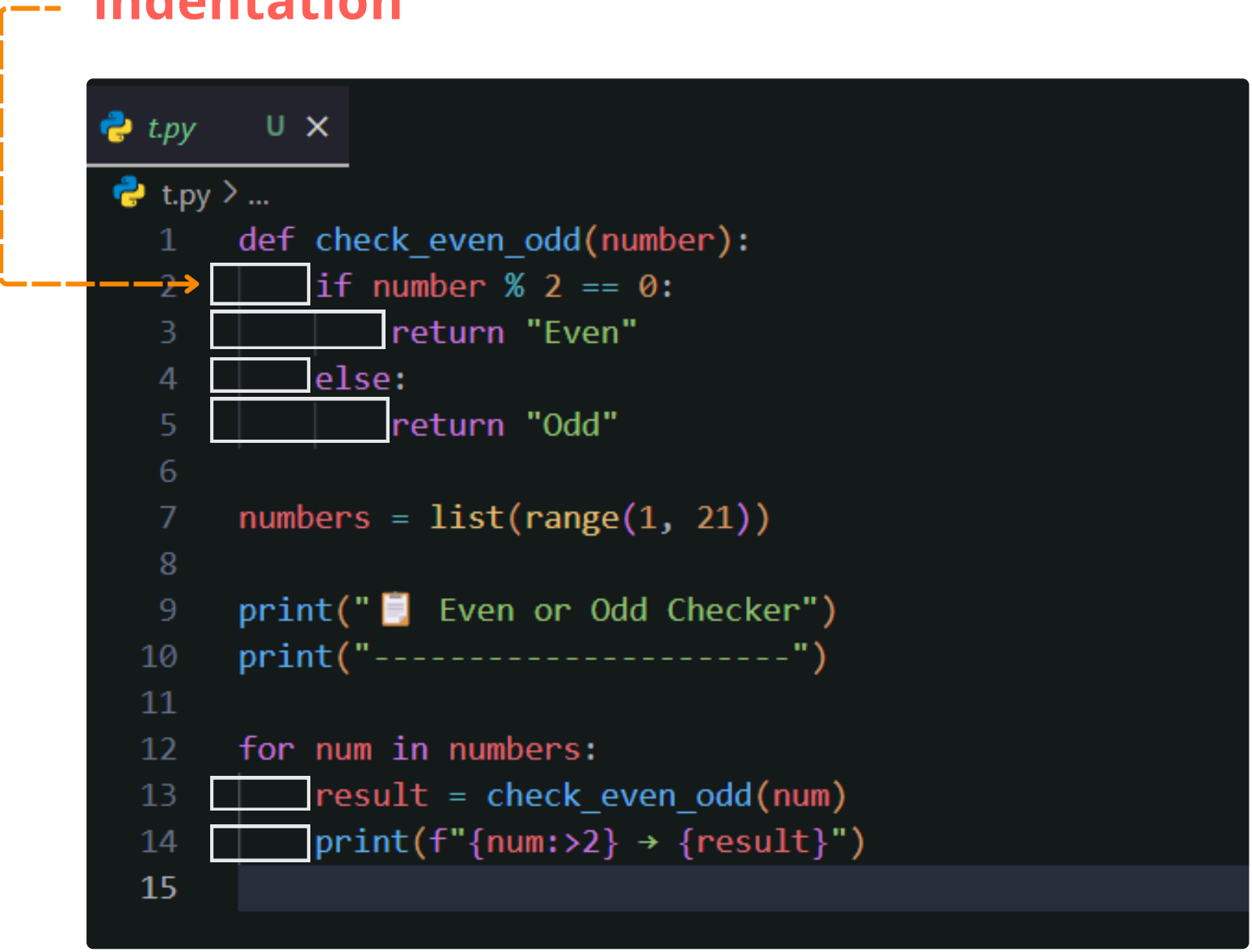
Key Differences Between Spaces and Tabs

Aspect	Spaces	Tabs
• Character Used	→ Uses multiple space characters	→ Uses a single tab character
• Visual Width	→ Fixed (usually 4 spaces)	→ Varies across editors
• PEP 8 Compliance	→ Recommended	→ Not recommended.
• Editor Compatibility	→ Consistent in all editors	→ May look different in each editor
• Risk of Mixing	→ Lower chance of errors	→ Can cause issues when mixed

How to Check Indentation Level in VS Code

- VS Code makes it easy to see what kind of indentation is being used.
- Normally in VS code or any Code editor you can see this blank spaces in your code which we called

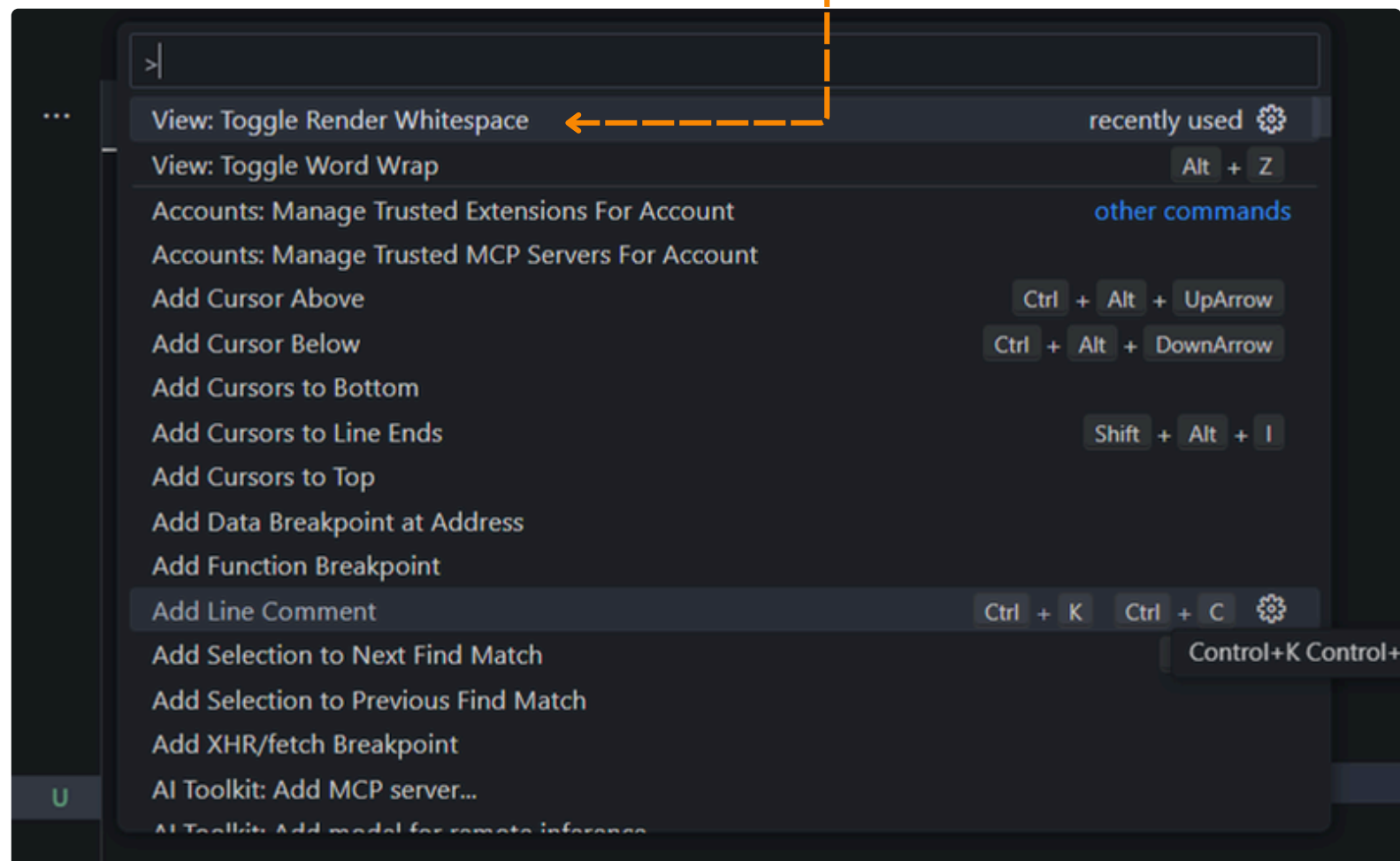
indentation



```
t.py  U X
t.py > ...
1  def check_even_odd(number):
2  [ ] if number % 2 == 0:
3  [ ] [ ] return "Even"
4  [ ] else:
5  [ ] [ ] return "Odd"
6
7  numbers = list(range(1, 21))
8
9  print(" 📄 Even or Odd Checker")
10 print("-----")
11
12 for num in numbers:
13 [ ] result = check_even_odd(num)
14 [ ] print(f"{num:>2} → {result}")
15
```

Option 1: Use Command Palette

- Press **Ctrl + Shift + P** (or **Cmd + Shift + P** on Mac)
- 🖱️ This opens the **Command Palette**
- Type and select **Toggle Render Whitespace**



After Toggle Render Whitespace

- Spaces will now show as dots (·).

A screenshot of a Python code editor window titled 't.py'. The code is as follows:

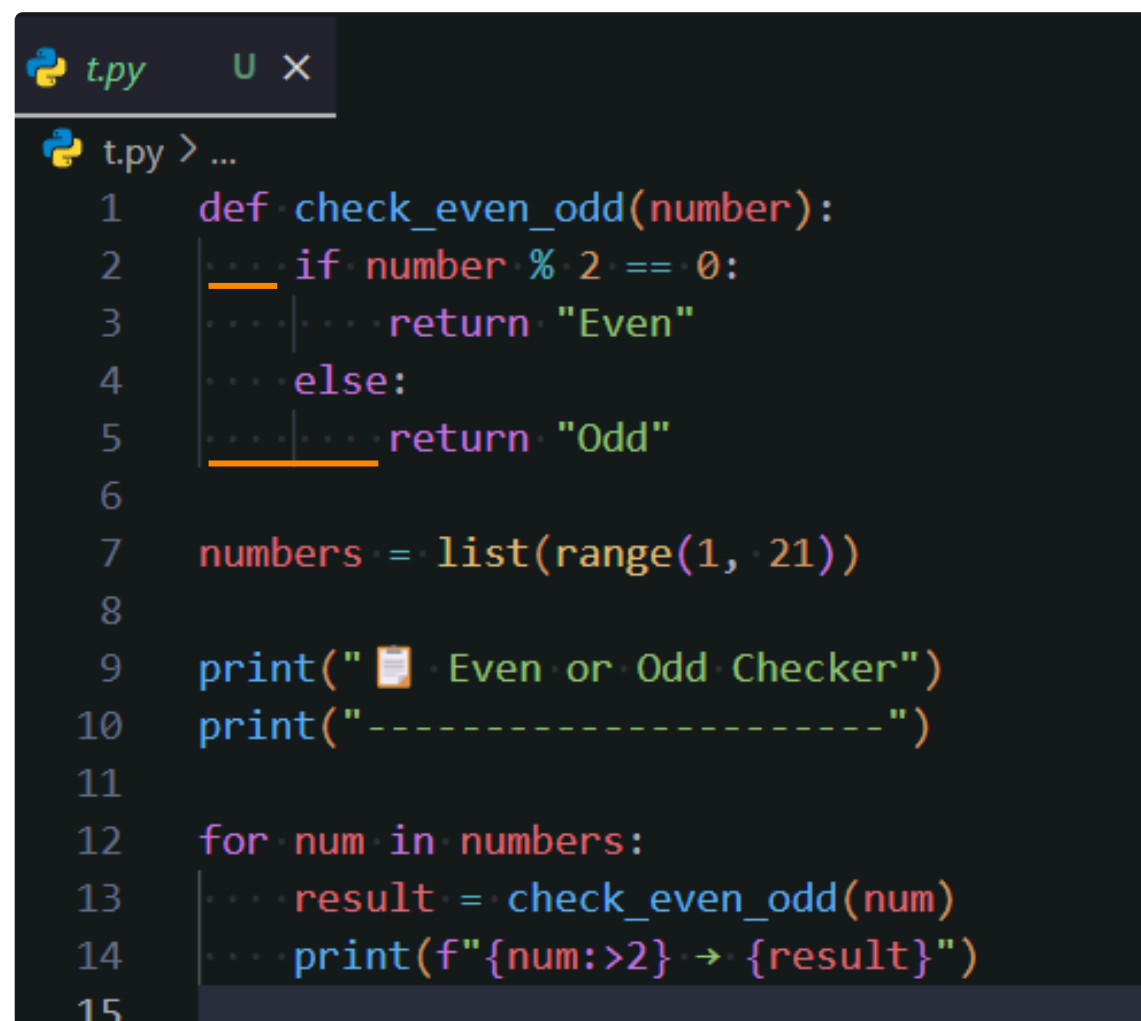
```
1 def check_even_odd(number):  
2     if number % 2 == 0:  
3         return "Even"  
4     else:  
5         return "Odd"  
6  
7 numbers = list(range(1, 21))  
8  
9 print(" 📄 Even or Odd Checker")  
10 print("-----")  
11  
12 for num in numbers:  
13     result = check_even_odd(num)  
14     print(f"{num:>2} → {result}")  
15
```

The code is formatted with standard Python indentation. The spaces between characters in the code are rendered as dots (·), making the code look like: `def check_even_odd(number):` becomes `def · check_even_odd(number):`. The output of the program is visible in the terminal area, showing the function results for each number in the range 1 to 20.

After Toggling "Render Whitespace" in VS Code

What you see depends on your existing indentation style:

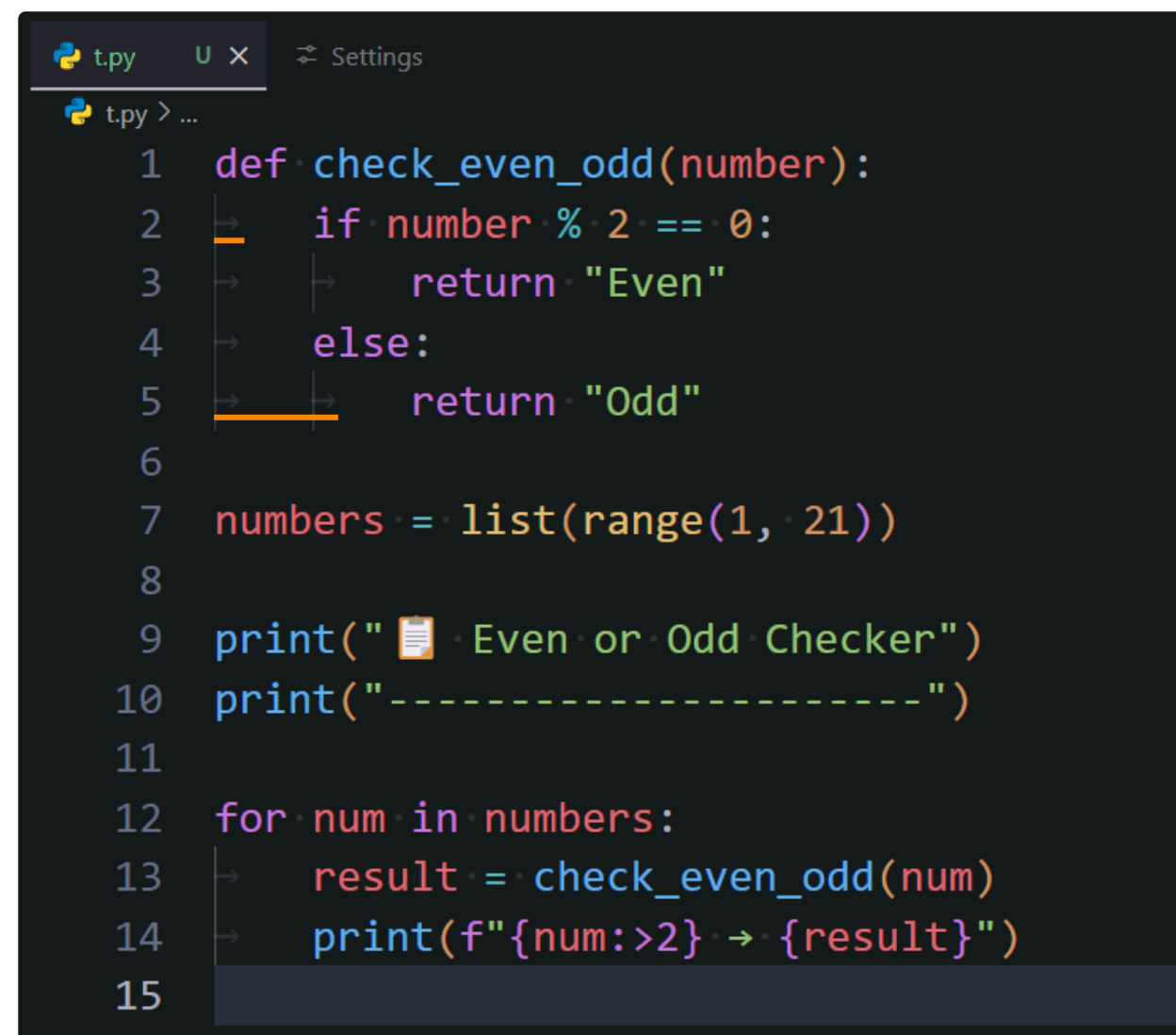
- If you're using **spaces**, they will appear as **dots (·)**.
 - If you're using **tabs**, they will appear as **arrows (→)**.
-
- **Spaces will now show as dots (·)**
 - **Tabs appear as arrows (→)**



The image shows a VS Code editor window with a file named 't.py'. The code is as follows:

```
t.py > ...
1 def check_even_odd(number):
2     if number % 2 == 0:
3         return "Even"
4     else:
5         return "Odd"
6
7 numbers = list(range(1, 21))
8
9 print(" 📄 Even or Odd Checker")
10 print("-----")
11
12 for num in numbers:
13     result = check_even_odd(num)
14     print(f"{num:>2} → {result}")
15
```

In this view, spaces are rendered as dots (·). For example, the first line is 'def check_even_odd(number):' and the second line is 'if number % 2 == 0:'. The indentation is shown as a series of dots.



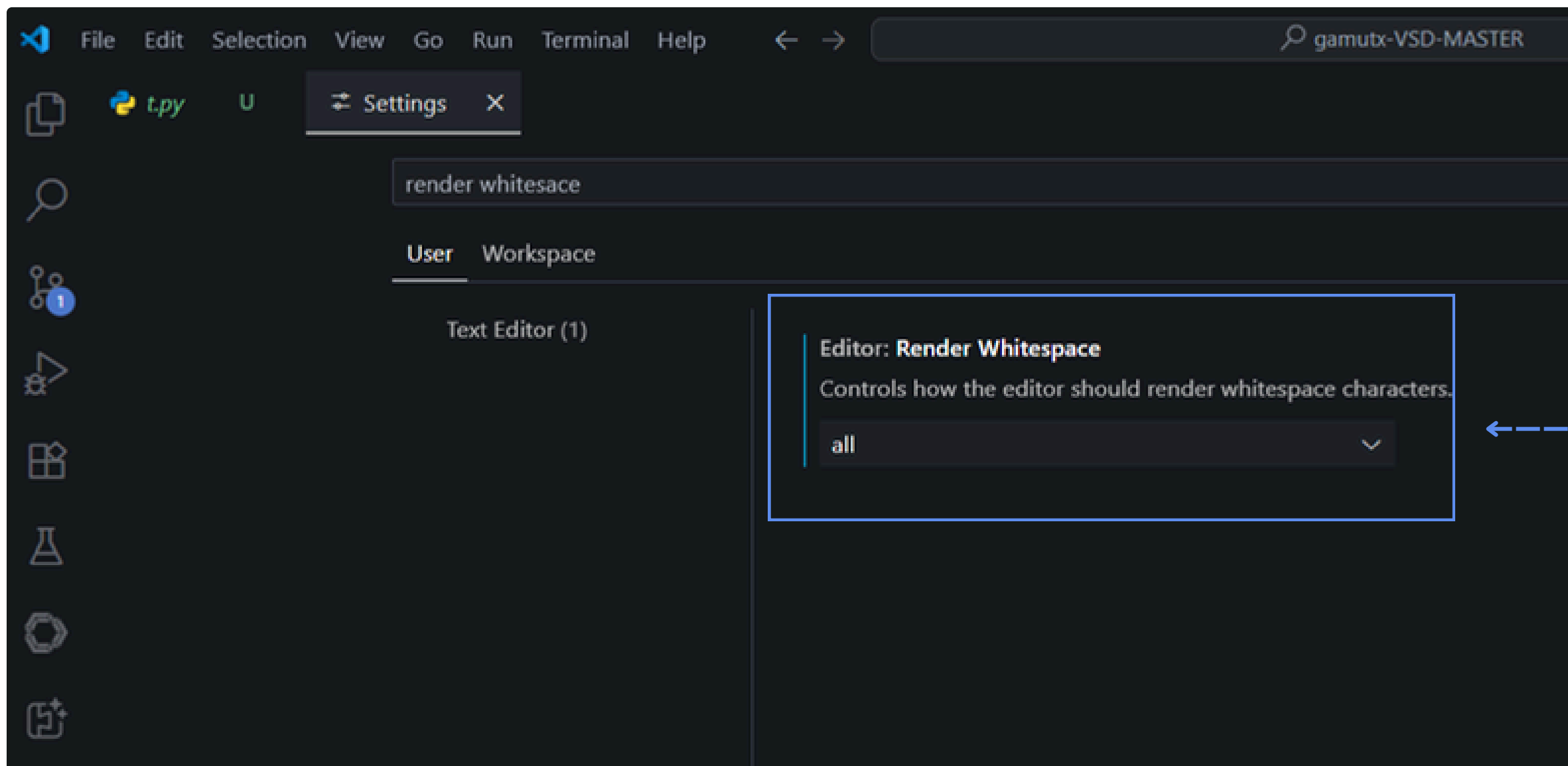
The image shows a VS Code editor window with a file named 't.py'. The code is as follows:

```
t.py > ...
1 def check_even_odd(number):
2     if number % 2 == 0:
3         return "Even"
4     else:
5         return "Odd"
6
7 numbers = list(range(1, 21))
8
9 print(" 📄 Even or Odd Checker")
10 print("-----")
11
12 for num in numbers:
13     result = check_even_odd(num)
14     print(f"{num:>2} → {result}")
15
```

In this view, tabs are rendered as arrows (→). For example, the first line is 'def check_even_odd(number):' and the second line is 'if number % 2 == 0:'. The indentation is shown as a series of arrows.

Option 2: Use Settings Menu

- Go to **Settings** (⚙️) (click the gear icon).
- Search for “**Render Whitespace**”.
- Set it to “**all**” so that whitespace is always visible.



Using Command Palette vs Using Setting Menu

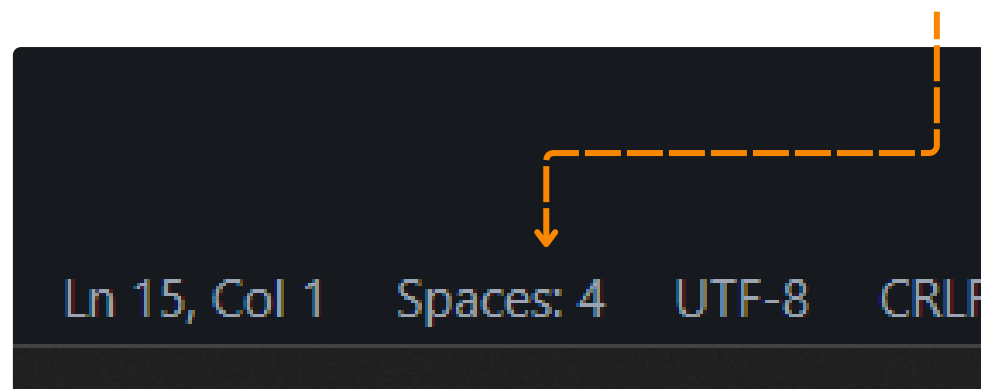
Method	Purpose	Visual Indicators	Best For
• Command Palette	→ Quickly show/hide whitespace	→ · = spaces → = tabs	→ Fast, temporary check
• Settings Menu	→ Always show whitespace in all files	→ · = spaces → = tabs	→ Ongoing visibility, all files

How to Set Indentation Level in VS Code

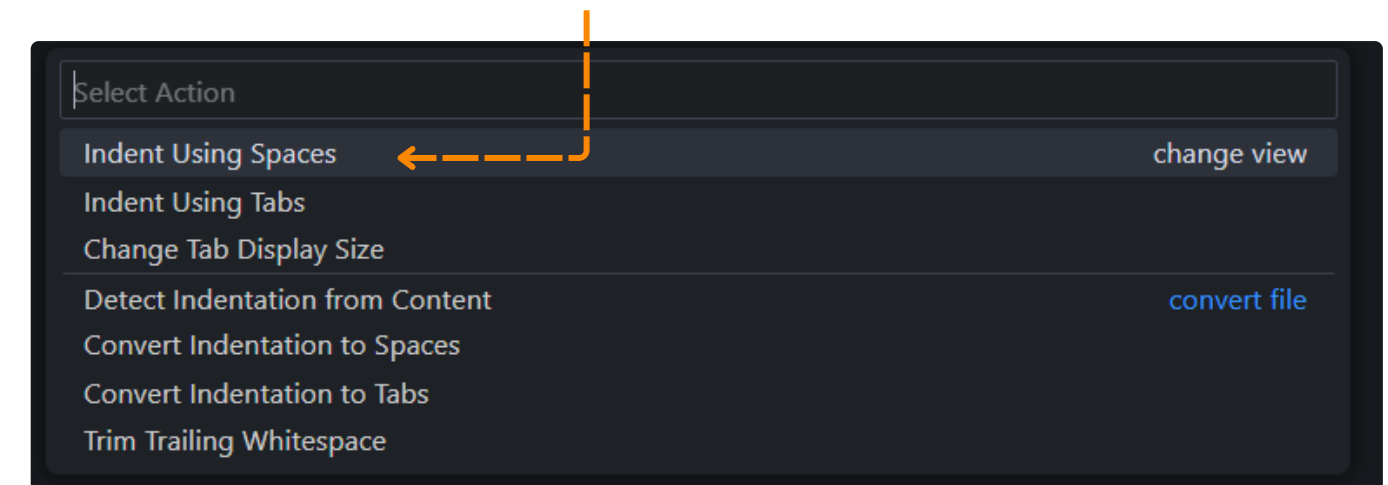
Python's standard is to use 4 spaces per indentation level — not tabs. Here's how to set that up.

Option 1: Use the Status Bar

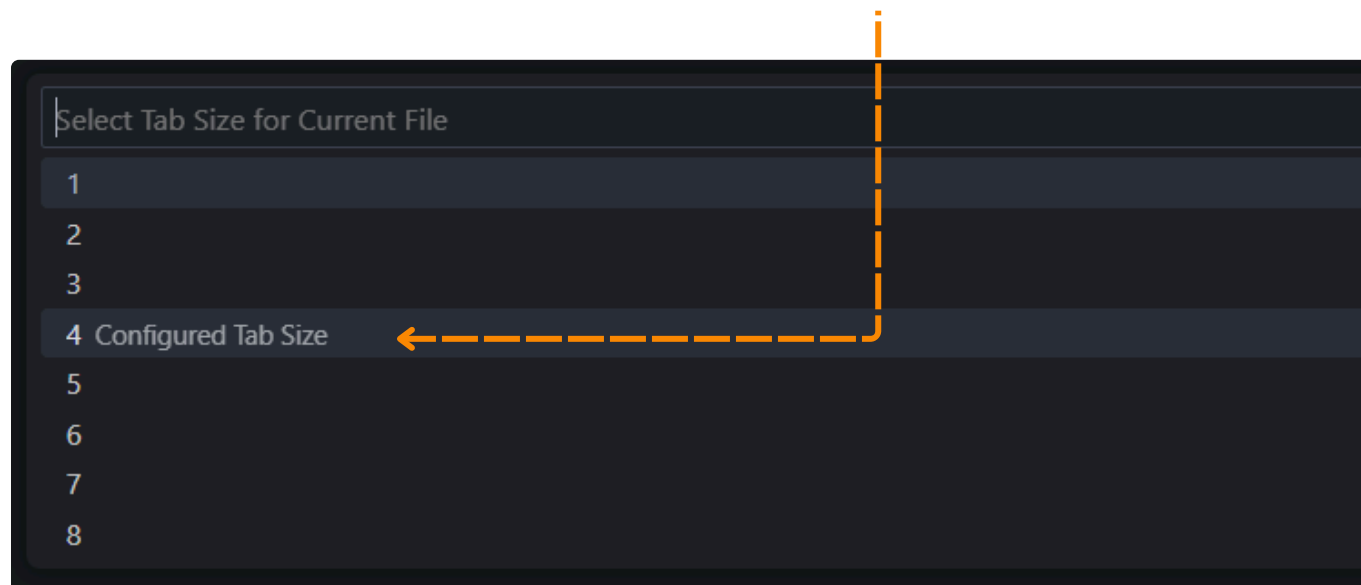
- Look at the bottom-right of your VS Code window.
- Click on the text that says "**Spaces: 4**" or "**Tab Size: X**".



- Choose "**Indent Using Spaces**"



- Set Tab/Indent **Size** to **4**.

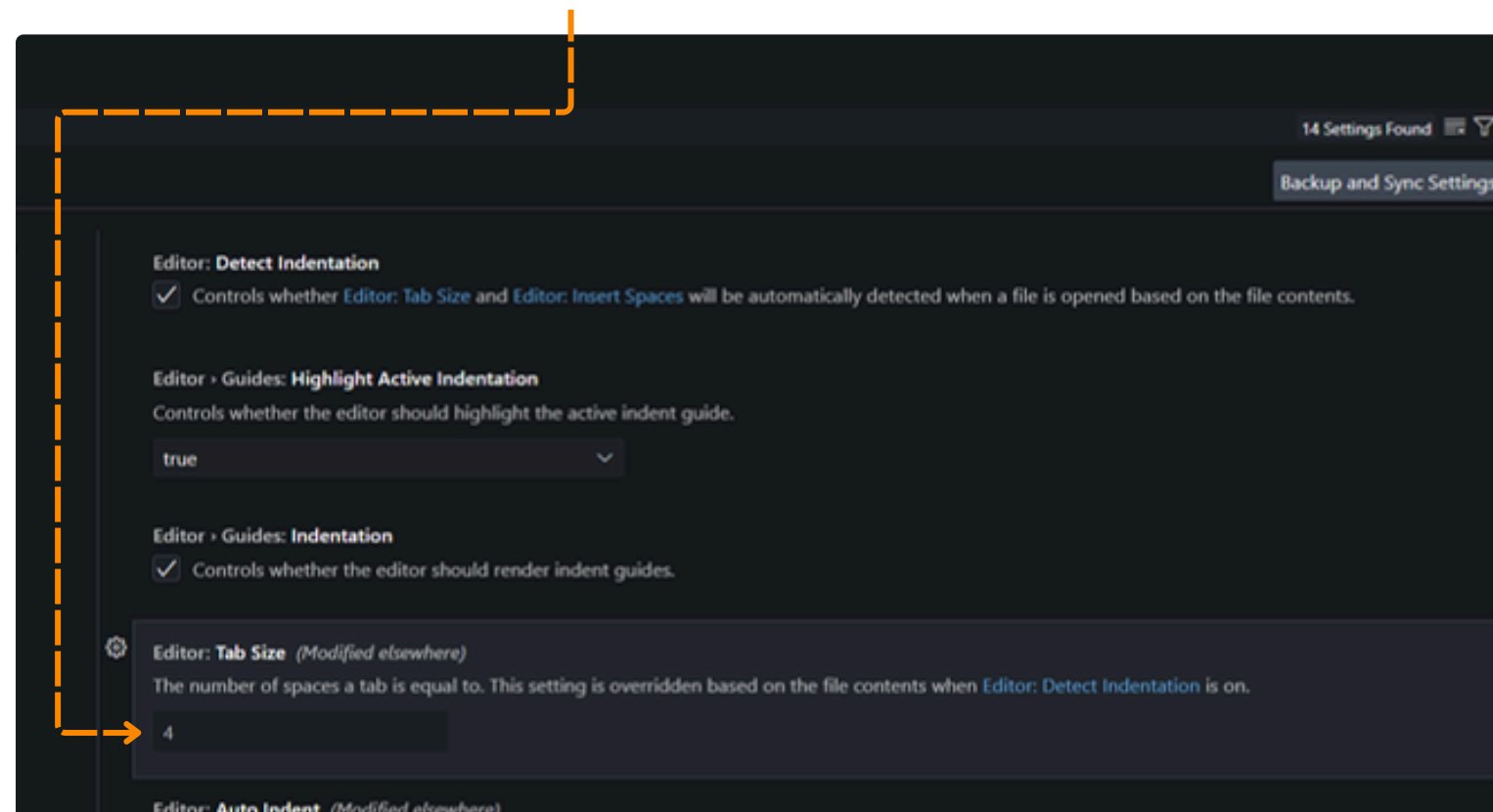


Option 2: Set Indentation permanent

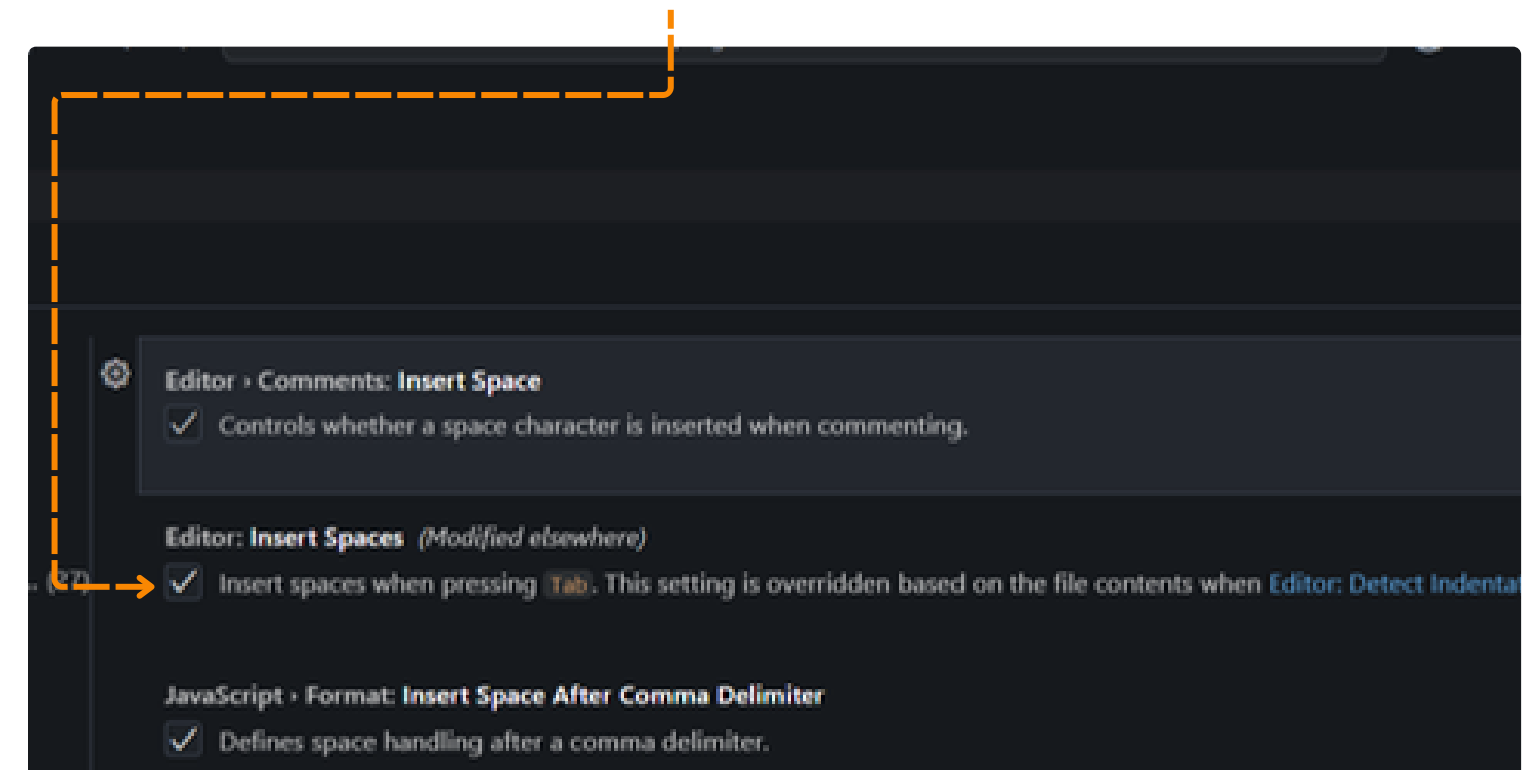
1. Click the gear icon in the bottom-left of VS Code.
2. Choose Settings.

- Do the Following 

- In the search bar at the top, type **indentation**.
- Set the Tab Size: **4**



- In the search bar at the top, type **insert Space**
- Make Sure the **insert Space** should be ticked



✓ (Make sure this is checked — this converts tabs to spaces)

Rules of Indentation in Python

1. Python Uses Indentation to Define Code Blocks

- Code blocks like **functions, loops, and conditionals** rely on indentation.
- There are **no {} brackets** in Python — indentation shows which lines belong together.

✓ **Example:**

```
if True:
    print("Condition is true")
```

2. Use 4 Spaces Per Indentation Level

- The standard convention in Python is to use **4 spaces (not a tab character)**.
- Most editors can be set to **insert 4 spaces** when you press Tab.

✓ **Example:**

```
def greet():
    print("Hello")
```

✗ **Avoid:**

```
def greet():
    print("Hello")
```

3. Don't Indent the First Line of Code

- The first line should begin at the **very start of the line**.
- This applies to **imports, function definitions, or variable assignments**.

✓ Example:

```
import os

def main():
    print("Running script")
```

✗ Incorrect:

```
    import os

    def main():
        print("Running script")
```

4. All Lines in a Block Must Be Indented the Same

- Every line inside a block (like inside a function or loop) must be aligned with the others.

✓ Example:

```
for i in range(3):
    → print(i)
    → print("Looping")
```

✗ Incorrect:

```
for i in range(3):
    → print(i)
    → print("Looping")
```

5. Don't Mix Tabs and Spaces

- Python treats tabs and spaces differently.
- **Mixing** them may not be visible, but it can cause **TabError**.

✗ **Incorrect:**

```
def hello():  
→print("This line uses a tab")  
····print("This line uses spaces")
```

- Use only **spaces** — configure your **editor** to convert tabs to spaces.

Common Mistakes in Conditional Statements

- Overlooking Edge Cases
- Making Sure Data Types Are Right
- Using Inefficient or Redundant Checks
- Not Considering All Possible Conditions

Overlooking Edge Cases

- This happens when **code doesn't account for uncommon or special scenarios**, such as:
 - **Empty inputs, Zero values, Extreme numbers**
- Ignoring edge cases **leads to bugs or unexpected behavior**

✗ Mistake

```
age = 0

if age > 18:
    print("You are an adult.")
else:
    print("You are not an adult.")
```

- This logic **assumes all non-adults are valid**
- **but what if age is 0 or negative?**

✓ Valid Code

```
age = 0

if age > 18:
    print("You are an adult.")

elif age > 0 and age < 18:
    print("You are not an adult.")

else:
    print("Invalid age.")
```

Now we've handled:

- Age **greater than 18** → Adult
- Age **between 1 and 17** → Not an adult
- Age **0 or less** → Invalid

Making Sure Data Types Are Right

- This error occurs when a **program assumes a value has the right data type**, but it doesn't.
- It can **cause**:
 - **Unexpected behavior**
 - **TypeError**s
 - **Confusing bugs**

✗ Mistake

```
user_input = "25"

if user_input > 18:
    print("You are an adult.")
else:
    print("You are not an adult.")

# ERROR!
# Traceback (most recent call last):
#   File "<main.py>", line 3, in <module>
# TypeError: '>' not supported between instances of 'str' and 'int'
```

- This will give a **TypeError**
- because we're **comparing a string ("25") to an integer (18)**.

✓ Valid Code

```
user_input = "25"
age = int(user_input)

if age > 18:
    print("You are an adult.")
else:
    print("You are not an adult.")

# You are an adult.
```

- Converting the string to int before comparing fixes the logic

Using Inefficient or Redundant Checks

- This error occurs when a **program assumes a value has the right data type**, but it doesn't. It can **cause**:
 - Unexpected behavior
 - TypeErrors
 - Confusing bugs

✗ Mistake

```
temperature = 75

if temperature >= 70:
    if temperature < 80:
        print("Comfortable temperature.")

# Comfortable temperature.
```

- This works, **but it uses two if statements when just one would be enough.**

✓ Valid Code

```
temperature = 75

if 70 <= temperature < 80:
    print("Comfortable temperature.")

# Comfortable temperature.
```

- This version is **shorter, cleaner**, and does the same thing using a **range check in one line.**

Not Considering All Possible Conditions

- This means **forgetting to check every possible situation in your code.**
- If you don't include all conditions, your **program might not work correctly.**
- **Adding an "else" statement helps cover all cases**, making your **code more complete and reliable.**

✗ Mistake

```
score = 85

if score >= 90:
    print("Excellent")
elif score >= 80:
    print("Good")

# Good
```

- **What about scores below 80?**
 - *The code says nothing in that case.*

✓ Valid Code

```
score = 85

if score >= 90:
    print("Excellent")
elif score >= 80:
    print("Good")
elif score >= 70:
    print("Average")
else:
    print("Needs Improvement")

# Good
```

- Now the code **handles every possible score**
- Including those **below 70**

Best Practices

- Avoiding Redundant Conditions
- Writing Readable and Maintainable Conditions
- Simplifying Complex Conditional Blocks

Avoiding Redundant Conditions

✗ Mistake

- $x > 5$ already means $x > 0$
- the second check is pointless.

```
x = 10

if x > 5 and x > 0:
    print("x is positive and greater than 5.")
```

✓ Valid Code

```
x = 10

if x > 5:
    print("x is greater than 5.")

# x is greater than 5.
```

- By removing the redundant check,
- The code remains clear and concise.
- The **condition $x > 5$** is sufficient to ensure that x is positive and greater than 5.

Writing Readable and Maintainable Conditions

✗ Mistake

- This works, but the condition can be hard to read at a glance.

```
age = 25

if age > 18 and age < 65:
    print("You are an adult.")

# You are an adult.
```

✓ Valid Code

```
age = 25
is_adult = age > 18 and age < 65

if is_adult:
    print("You are an adult.")

# You are an adult.
```

- Using a **variable** like `is_adult` makes the code **easier** to understand.
- This is **helpful when working in teams** or coming back to the code later.

Simplifying Complex Conditional Blocks

✗ Mistake

```
temperature = 70
is_raining = False
is_snowing = False

if temperature < 50:
    if is_raining:
        print("It's cold and rainy.")
    elif is_snowing:
        print("It's cold and snowy.")
    else:
        print("It's cold but not rainy or snowy.")
elif temperature < 70:
    if is_raining:
        print("It's cool and rainy.")
    elif is_snowing:
        print("It's cool and snowy.")
    else:
        print("It's cool but not rainy or snowy.")
else:
    print("The weather is warm.")

# The weather is warm.
```

- This code works, **but it's long and harder to read.**

✓ Valid Code

```
temperature = 70
is_raining = False
is_snowing = False

if temperature < 50:
    weather = "cold"
elif temperature < 70:
    weather = "cool"
else:
    weather = "warm"

if is_raining:
    weather += " and rainy"
elif is_snowing:
    weather += " and snowy"

print(f"The weather is {weather}.")

# The weather is warm.
```

- **Shorter and more organized**
- Easier to understand and modify

Thank You

